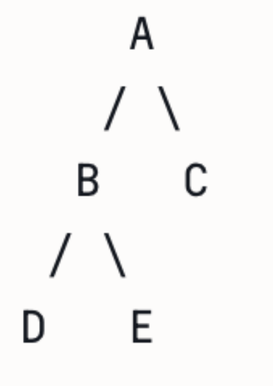
Introduction to Parallel & Distributed Programming

Lec 26— Parallel Algorithms

Subodh Sharma | April 24, 2026

Parallel DFS

- DFS inherently difficult to parallelise
  - Depth exploration — creating long sequential chains
- Constructing a valid **DFS tree** — quite hard to parallelise
- Constructing a specific DFS order — like the smallest neighbour first
  - Even harder!
- Naive 1st attempt: Explore all neighbours (B,C) simultaneously
  - The discovery pattern is no longer **Depth-first;** it is just parallel traversal



Parallel DFS

Work Pool Parallel Search

- Parallel **depth-biased** search
  - Not guaranteed to output a strict DFS tree, but a good start.
- Algorithm: Maintain a shared stack (or bag) of frontier vertices
  - Many processors repeatedly do the following:
    - Remove vertex u from the bag
    - Inspect its adjacency list
    - For each unvisited neighbour v, try to mark it as visited
    - If successful, set $parent[v] = u$ and insert v in the bag

Parallel DFS

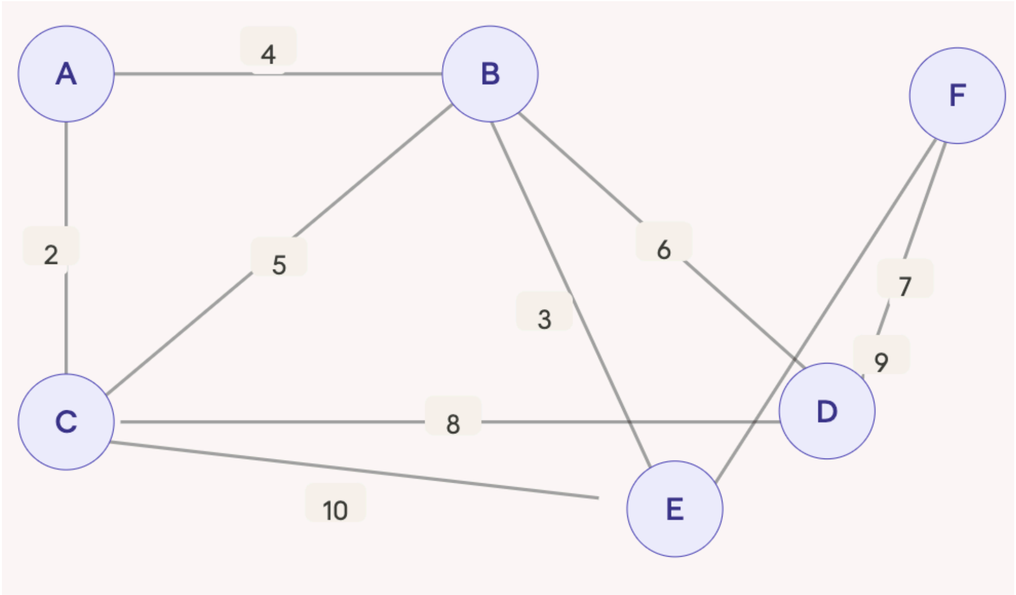
Divide & Conquer

- High level steps:
  - Identify a path that acts as a spine of the DFS tree
  - Find connected components hanging off the path
  - Assign each component to an attachment vertex on the spine
  - Recursively compute DFS Trees of the components in parallel
  - Merge recursive trees into one global DFS tree
  - Verify DFS property, if needed

Parallel DFS

Divide & Conquer

- High level steps:
  - **Identify a path that acts as a spine of the DFS tree**
    - Step 1: compute a spanning tree S (parallel Kruskal's algorithm)



Parallel Kruskal's Algorithm

- Parallel sort the edges by edge weight
  - Can use parallel merge sort. Local chunks can be sorted in (E/p log E/p) time
  - Merge reduction combines them in O(log p) time
  - Ex: [38, 27, 43, 3, 9, 82, 10, 19]
  - $T_1 : [38,27,43,3] --> [3,27,38,43],$
  - $T_2 : [9,82,10,19] --> [9,10,19,82]$
  - Parallel Merge:
    - Pick the median of the larger array (let's say it is A[mid])
    - Binary search to find the position in the other array B where A[mid] would fit. Let us call this index k
    - Now you have four segments: Left side from A[mid], Right Side from [Amid], Left side from B[k], Right side from B[k]
      - Concurrent execution: Processor1 recursively merge left side of A with left side of B and Processor 2 recursively merges right side of A with right side of B

Complexity Derivation

Parallel Kruskal's Algorithm

- For each component, run **Boruvka rounds**
  - Every component finds its minimum-weight outgoing edge
  - Selected edges are de-duplicated (two components may pick the same edge)
  - Components are contracted in to **super-vertices**
  - **Eg:** On the same example as on slide 9

Parallel Kruskal's Algorithm

Processor	Component	All incident edges (sorted by w)	Min outgoing edge chosen
P_1	A	(A,C) w=2 (A,B) w=4	✓ (A,C) w=2
P_2	B	(B,E) w=3 (A,B) w=4 (B,C) w=5 (B,D) w=6	✓ (B,E) w=3
P_3	C	(A,C) w=2 (B,C) w=5 (C,D) w=8 (C,E) w=10	✓ (A,C) w=2
P_4	D	(B,D) w=6 (D,F) w=7 (C,D) w=8	✓ (B,D) w=6
P_5	E	(B,E) w=3 (E,F) w=9 (C,E) w=10	✓ (B,E) w=3
P_6	F	(D,F) w=7 (E,F) w=9	✓ (D,F) w=7

- Notice (A,C) and (B,E) appear twice — Deduplication using a parallel scan

(A,C) w=2 — from P_1 | (B,E) w=3 — from P_2 | (A,C) w=2 — from P_3 ← duplicate!

(B,D) w=6 — from P_4 | (B,E) w=3 — from P_5 ← duplicate! | (D,F) w=7 — from P_6

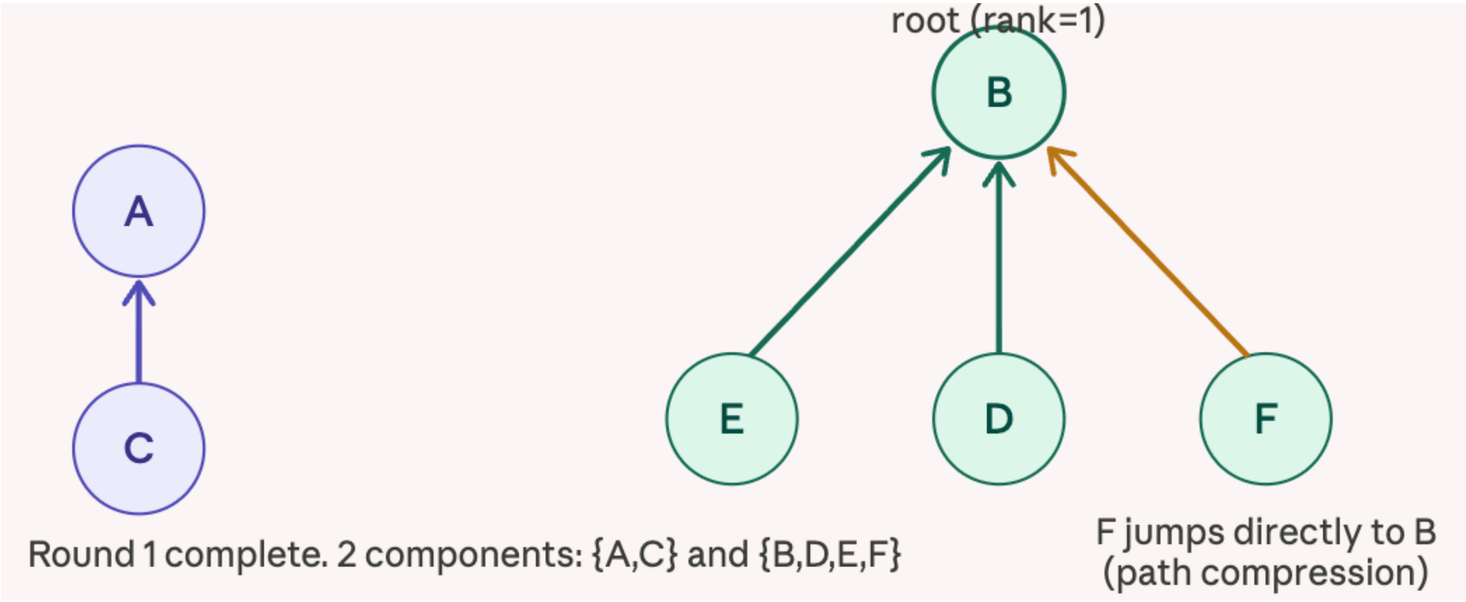
Parallel Kruskal's Algorithm

- For each component, run **Boruvka rounds**
  - Every component finds its minimum-weight outgoing edge
  - Selected edges are de-duplicated (two components may pick the same edge)
  - Components are contracted in to **super-vertices**
  - **Eg:** On the same example as on slide 9

Parallel Kruskal's Algorithm

Parallel Union Find

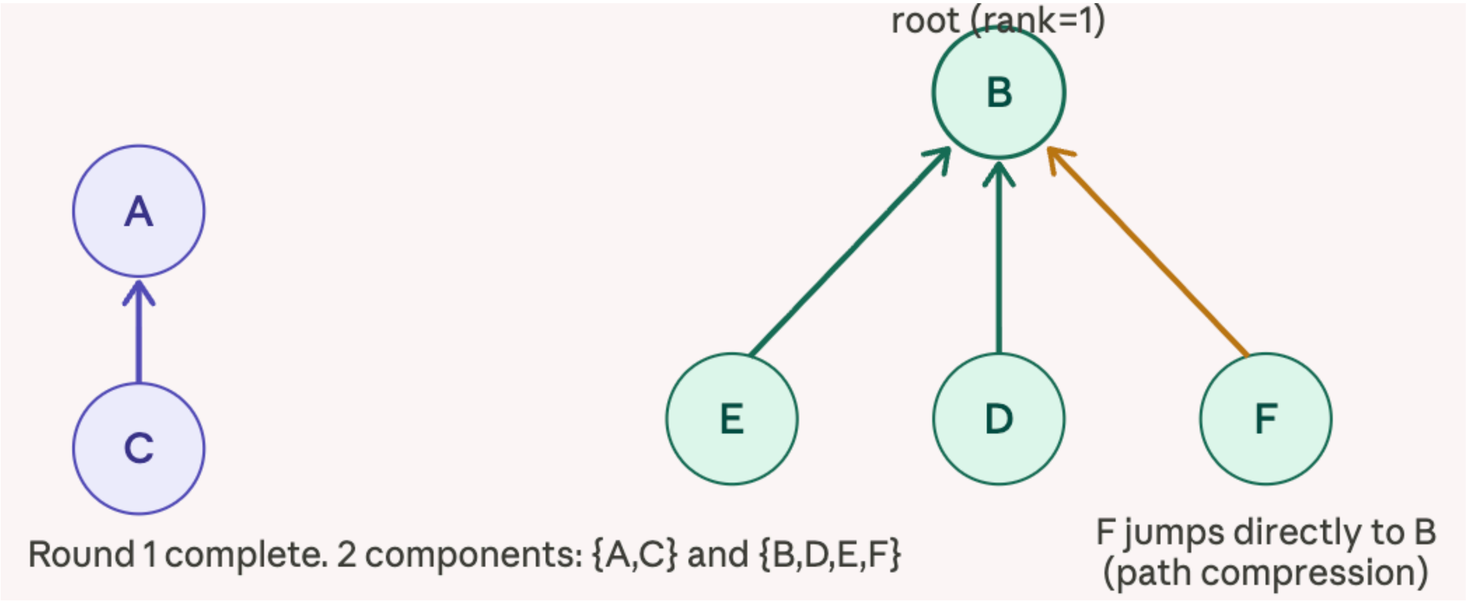
- For 4 de-duplicated edges: (A,C), (B,E), (B,D) and (D, F)
  - Run a parallel Union-Find with pointer jumping
  - First each processor handles one edge and calls $Union(u, v)$
  - Notice the compression in the case of edge (D,F)
  - Now the two component need to be connected.

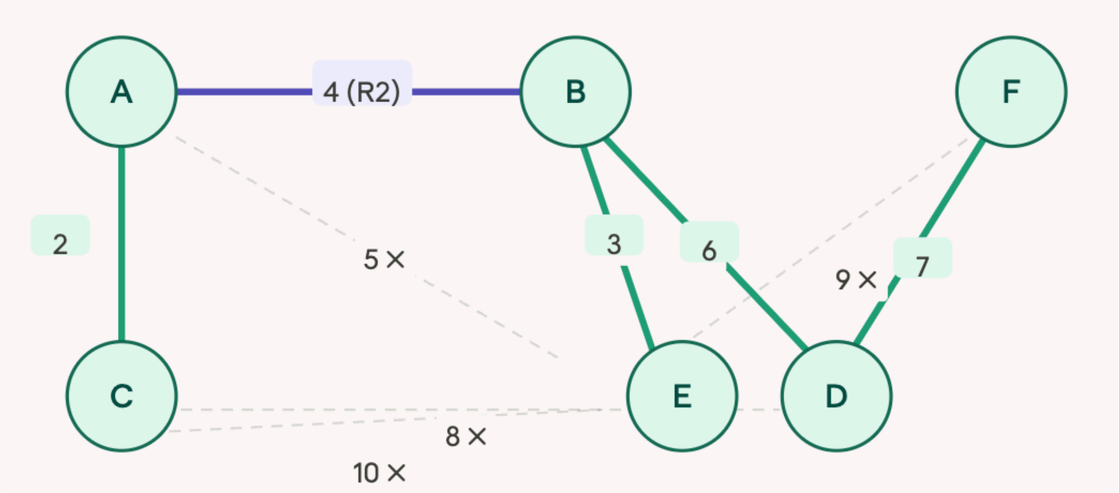


Parallel Kruskal's Algorithm

Cross Component Edge Detection

- For 4 de-duplicated edges: (A,C), (B,E), (B,D) and (D, F)
  - Now the two component need to be connected.
    - Min edge selection that crosses the components
    - Each processor looks at one edge and checks in O(1) whether its cross-component
      - Does by checking $Find(u) \neq Find(v)$
      - And if so, what is its weight
    - Parallel min reduction on the collected list of cross-edges





Parallel DFS

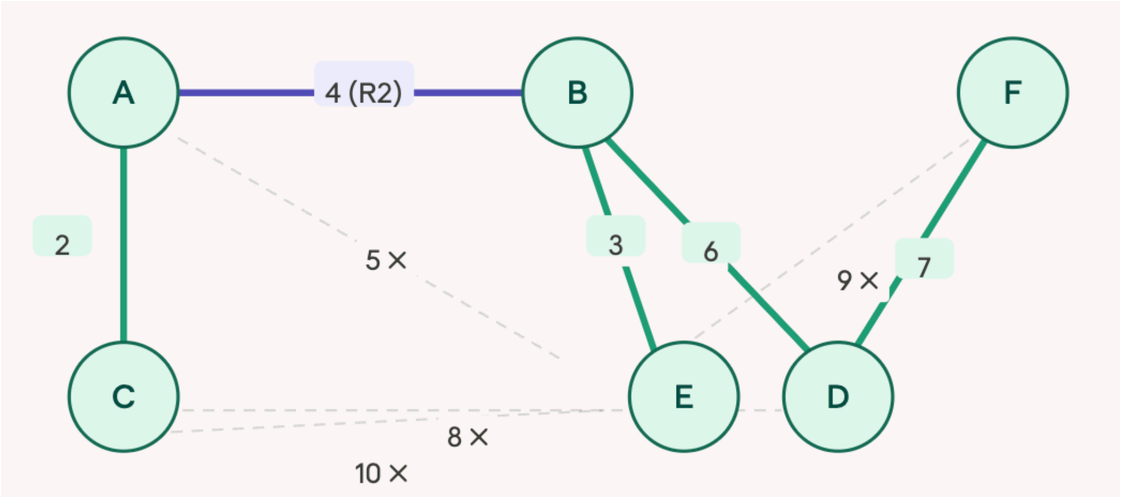
Divide & Conquer

- High level steps:
  - **Identify a path that acts as a spine of the DFS tree**
    - **Step 1: compute a spanning tree S (parallel Kruskal's algorithm)**
    - Step 2: Extract root-to-leaf path

Extracting Root-to-leaf Path

- Step 1: Compute an Euler Tour in parallel
  - Replace each undirected tree edge (u, v) with two directed arcs: $u \rightarrow v; v \rightarrow u$
    - For each arc, we also compute its $next$ pointer — ie, the arc that follows it in the DFS traversal order of the tree
      - Rule 1: $next(u \rightarrow v)$ is the first arc out of v going to v's first child. If v is a leaf then $next(u \rightarrow v) = v \rightarrow u$ (and returns immediately)
      - Rule 2: $next(v \rightarrow u)$ is the next arc out of u, ie $u \rightarrow next_child_of_u_after_v$; if v was the last child of u, then $next(v \rightarrow u) = (u \rightarrow parent(u))$
        - If the root's last child's return arc, $next$ wrap backs to the first arc out of root, then the tour ends!

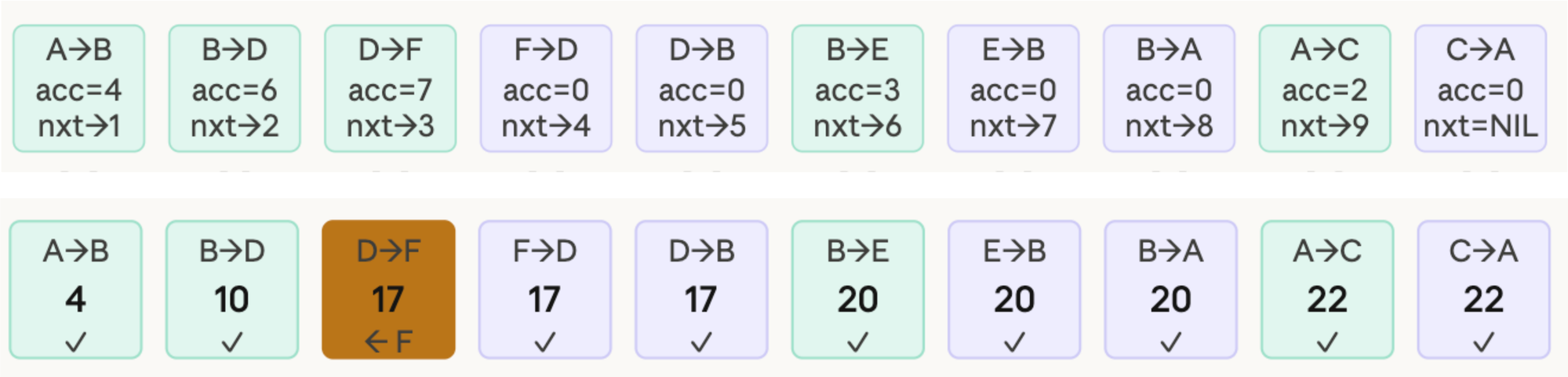
Extracting Root-to-leaf Path



pos 0	pos 1	pos 2	pos 3	pos 4	pos 5	pos 6	pos 7	pos 8	pos 9
A→B ↓ down	B→D ↓ down	D→F ↓ down	F→D ↑ up	D→B ↑ up	B→E ↓ down	E→B ↑ up	B→A ↑ up	A→C ↓ down	C→A ↑ up

Extracting Root-to-leaf Path

- **Step 1: Compute an Euler Tour in parallel**
- Step 2: Parallel List Ranking via parallel prefix sum
  - Eg:

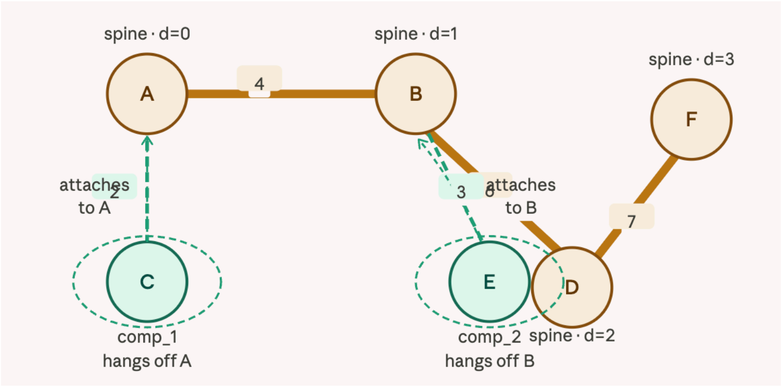


  - Reconstruct the path from leaves to root in parallel and pick the path with the deepest leaf: $A \rightarrow B \rightarrow D \rightarrow F$

Parallel DFS

Divide & Conquer

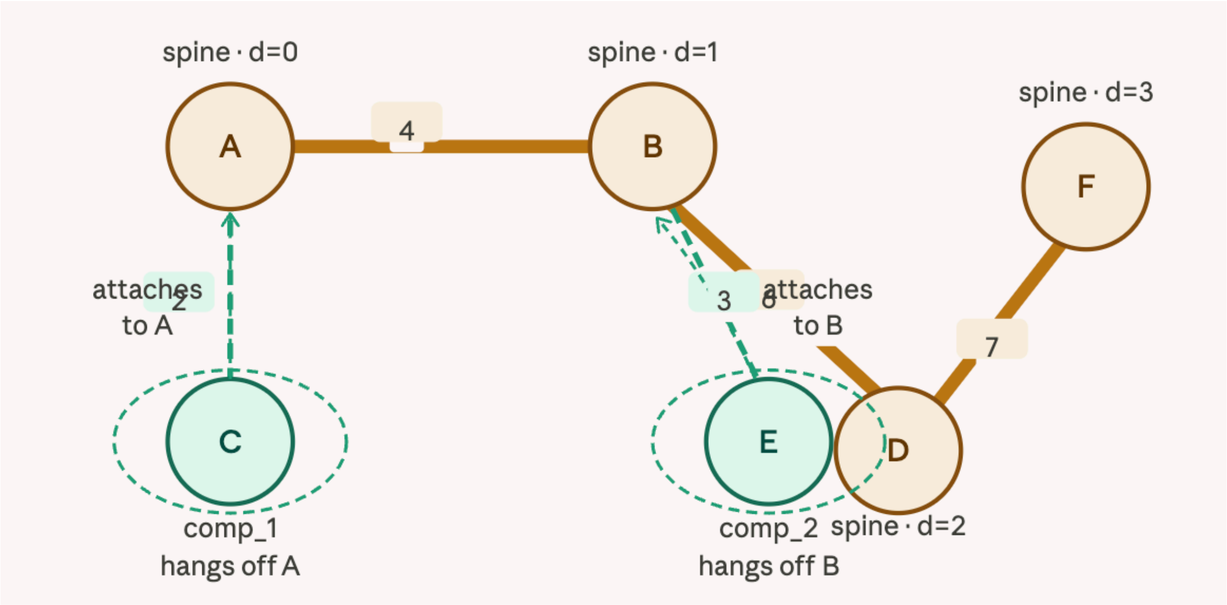
- High level steps:
  - **Identify a path that acts as a spine of the DFS tree**
    - **Remove/Contract all the vertices of this path from the graph**
  - Find connected components hanging off the path
    - Use the parallel Union-find using pointer jumping
      - For each off-spine node v: $off_parent[v] = parent[v]$ if $parent[v]$ is also off-spine
      - Else:$off_parent[v] = v$
    - At each round every processor is simultaneously executing: $off_parent[v] = off_parent[off_parent[v]]$



Parallel DFS

Divide & Conquer

- High level steps:
  - **Identify a path that acts as a spine of the DFS tree**
  - Find connected components hanging off the path



Parallel DFS

Divide & Conquer

- High level steps:
  - **Identify a path P that acts as a spine of the DFS tree**
  - **Find connected components hanging off the path P**
  - Assign each component to an attachment vertex on the spine
    - We use a simple deterministic rule: Assign component C_i to the deepest spine vertex in P
    - Algorithm:
      - Inspect all edges $(u, v_j), u \in C_i, v_j \in P$
      - Compute the maximum such j
      - Attach C_i to v_j

Parallel DFS

Divide & Conquer

- High level steps:
  - **Identify a path that acts as a spine of the DFS tree**
  - **Find connected components hanging off the path**
  - **Assign each component to an attachment vertex on the spine**
  - Recursively compute DFS Trees of the components in parallel
    - Each component C_i rooted at v_j becomes a recursive graph problem
      - For each C_i, with $root_i$ being the first entry vertex in C_i we invoke DFS(C_i, $root_i$)
        - The recursive calls are independent and run in parallel
      - One can choose deterministically $root_i$ from the first edge in the adjacency list, or the smallest endpoint label

Parallel DFS

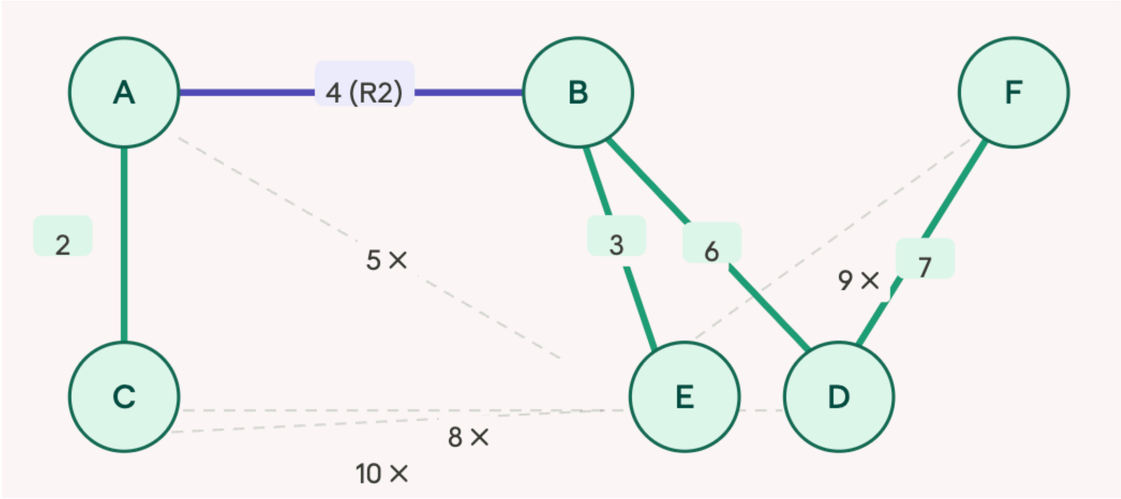
Divide & Conquer

- High level steps:
  - **Identify a path that acts as a spine of the DFS tree**
  - **Find connected components hanging off the path**
  - **Assign each component to an attachment vertex on the spine**
  - **Recursively compute DFS Trees of the components in parallel**
  - Merge recursive trees into one global DFS tree
  -

Parallel Tree Merging

- A DFS Tree merge must satisfy three invariants:
  - **I1**: Attachment edge $(Spine_node \rightarrow C_i_root)$ becomes a tree edge in the merged DFS tree
  - **I2**: DFS numbering respects pre-order: spine node s numbered before all nodes in its hanging component
  - **I3**: Non-tree crossing edges become back-edges or cross-edges classified by DFS number intervals

Parallel Tree Merging



- What we have:

Spine P — path A→B→D→F

Node	spine_pos	local DFS#
A	0 (root)	0
B	1	1
D	2	2
F	3 (leaf)	3

T_1 — DFS tree of comp_1 = {C}

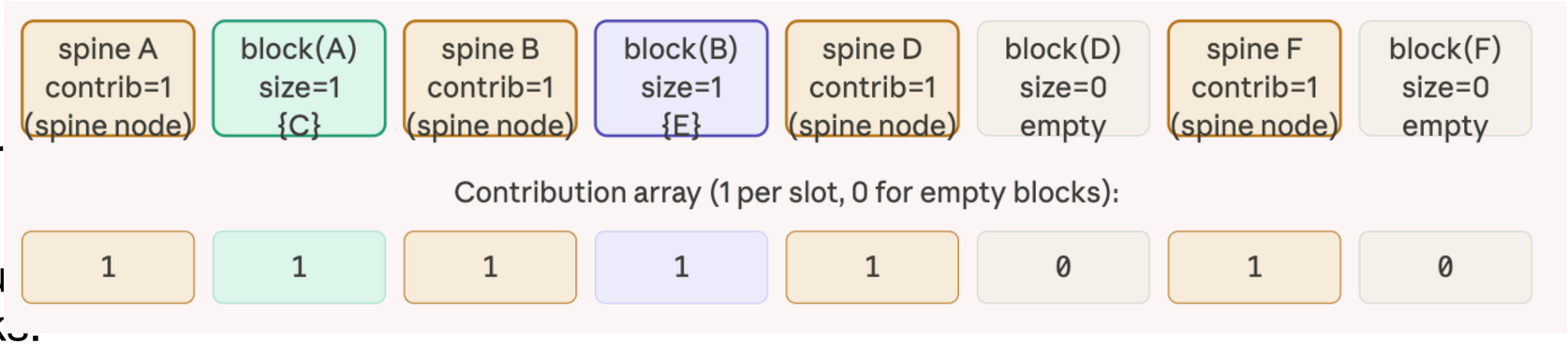
Node	parent in T_1	local DFS#	subtree size
C	— (root of T_1)	0	1

T_2 — DFS tree of comp_2 = {E}

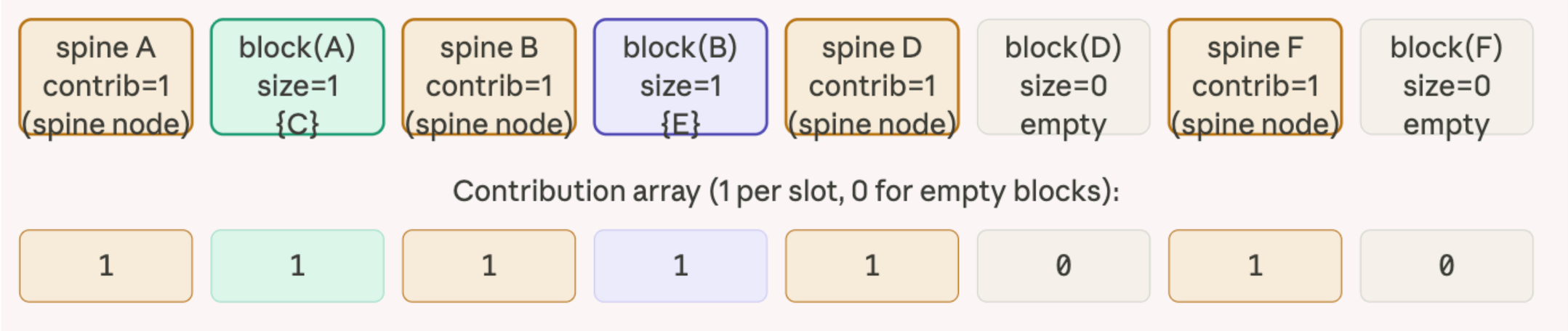
Node	parent in T_2	local DFS#	subtree size
E	— (root of T_2)	0	1

Parallel Tree Merging

- **Step1**: What we want:
  - Size of component C_i hanging off s
    - Parallel reduction in $O(log_2 n)$
  - The global DFS num
    - Global DFS number
      - This can be compu
        their hanging block



Parallel Tree Merging



Apply parallel prefix sum on this array → gives each slot its global DFS start number

Parallel Tree Merging

- **Step 2**: Tag all non-tree edges as **back-edge**, **forward-edge**, or **cross-edge** in the merged DFS tree.
  - For a non-tree edge (u, v):
    - If $dfs[u] < dfs[v]$ and $dfs[v] \leq dfs[u] + size[u] - 1$: then **forward edge**
    - If $dfs[v] < dfs[u]$, and $dfs[u] \leq dfs[v] + size[v] - 1$: then **back edge** (u is the descendant of v)
    - Else **cross edge**

Thank You!